

Scan to follow along

Open today's slides on your own phone or laptop — then you can run real code yourself.

hassanmiir.github.io/cpp-course/lecture1

Dr. Mir Hassan

Associate Professor in Artificial Intelligence
and Trustworthy Digital Systems



Point your camera here



0

opened today

Welcome to Software Engineering Hands-On Class

Today you become a **programmer** — write your very first real program, live, together.



Real C++



Run it live



Build a project



Together

A 2-hour hands-on session · no experience needed · just curiosity

WELCOME

You are about to become a programmer

By the end of **today**, you will have written, changed, and run real programs — right here in these slides.

By the end of the **course**, you will have built a complete, working application of your own.

**No experience assumed**

If you've never written a line of code, you are exactly who this course is for.

COURSE OUTLINE

The full plan — 10 lectures

**1 • First Steps**

Your first C++ program

**3 • Operators**

Maths & comparisons

**5 • Loops**

Repeating work

**7 • Arrays & Vectors**

Collections of data

**9 • Structs**

Grouping data

**2 • Variables**

Storing & naming data

**4 • Decisions**

if / else logic

**6 • Functions**

Reusable blocks

**8 • Strings**

Working with text

**10 • Files & Project**

Save data & finish

One step at a time

Each lecture adds one new idea — and one piece of the project we build together.

Where we're going — all 10 lectures

- 01  **First Steps** — your first program **YOU ARE HERE**
- 02  **Variables** — storing and naming information
- 03  **Operators** — arithmetic and comparisons
- 04  **Decisions** — if / else, choosing a path
- 05  **Loops** — repeating work automatically

...and the second half

- 06  **Functions** — naming and reusing blocks of logic
- 07  **Arrays & Vectors** — collections of values
- 08  **Strings** — reading and transforming text
- 09  **Structs** — grouping related data together
- 10  **Files** — saving data & finishing your project

THE DESTINATION

We'll build a real project — *together*

Every lecture adds one piece to a working program. **You** help decide what we build. Today we'll brainstorm ideas; here are two to spark your thinking:

**e.g. Study Buddy**

A tool **for students**: track subjects, log study hours, record grades, see your progress.

**e.g. Teacher Gradebook**

A tool **for teachers**: manage a class, record scores, compute averages, find top students.

Your idea counts

A game? A quiz app? A budget tracker? We'll pick our project as a class and build it step by step.

HOW TODAY WORKS

Watch a little, code a lot

**See**

Short explanations with clear examples — never long lectures.

**Try**

Live code boxes right in the slides. You edit, you run, you see output.

**Build**

Small challenges and a mini-project you finish with a partner.

Look for this

Whenever you see a dark box with a green ▶ **Run** button, that's your turn — type and run real C++.

Three rules for a great class



Ask anything, anytime

If you're confused, five others are too. There are no silly questions here.



Errors are normal and good

Every programmer sees errors constantly. They are hints, not failures.



Help the person next to you

Explaining something is the fastest way to truly learn it.

PART 1

What is programming, really?

Let's build the idea from the ground up — no jargon.



A program = precise instructions



A language + compiler makes it run



Why C++ is worth learning



THE BIG IDEA

A program is a set of instructions

A **program** tells a computer exactly what to do — step by step, in order.

Everyday analogy

A recipe is a program: "Add flour, then stir, then bake 20 min." The cook follows each step in order.

The computer is fast and obedient — but it does *exactly* what you say, not what you meant.



THE BIG IDEA

Why a *language*?

Computers don't understand English. A **programming language** is a precise way to write instructions a machine can follow.

**You write**

Human-readable code, in C++.

**Compiler translates**

Into instructions the machine runs.

**Computer runs**

Executes them, step by step.

Three steps, every time



1 • Write

Save your code in a `.cpp` file.



2 • Compile

Translate it into an executable.



3 • Run

Launch it and see the output.

The good news

Today the slides do the compile step *for you* — just click ► **Run**.

WHY THIS LANGUAGE

Why learn C++?

**Fast and powerful**

Games, operating systems, browsers, embedded devices, scientific computing.

**Teaches how computers really work**

Concepts here transfer to *every* other language.

**Strong foundations**

After C++, Java, C#, and Python feel familiar.

QUICK THOUGHT

Which of these use C++?

Tap the answer you think is right:

A Video games & game engines

B Web browsers like Chrome

C Car & aircraft systems

D All of the above

PART 2

Your first C++ program

Five lines — then we take each one apart, together.



Write & run "Hello, World!"



The anatomy: include, main, cout, ;



Your first live code — you run it!



YOUR FIRST PROGRAM

"Hello, World!"

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello, World!" << std::endl;
5     return 0;
6 }
```

hello.cpp

When you run this

the computer prints `Hello, World!` on the screen. Let's understand every piece.

The `#include` line

```
#include <iostream>
```

This loads a **library** — ready-made code. `iostream` gives us **input** and **output** tools (the "io").

Like saying: "I'll use the screen and keyboard, so load those tools first."

The `main` function

```
1 int main() {  
2     // your instructions go here  
3 }
```

Every program starts at `main` — the front door. The braces `{ }` hold everything it does.

Printing with `cout`

```
std::cout << "Hello, World!" << std::endl;
```

`std::cout`

The output stream — the screen.

`<<`

Sends what's on the right into cout.

`std::endl`

Ends the line (newline after).

Two vital small rules



return 0; means "finished successfully"

Tells the system the program ended with no errors.

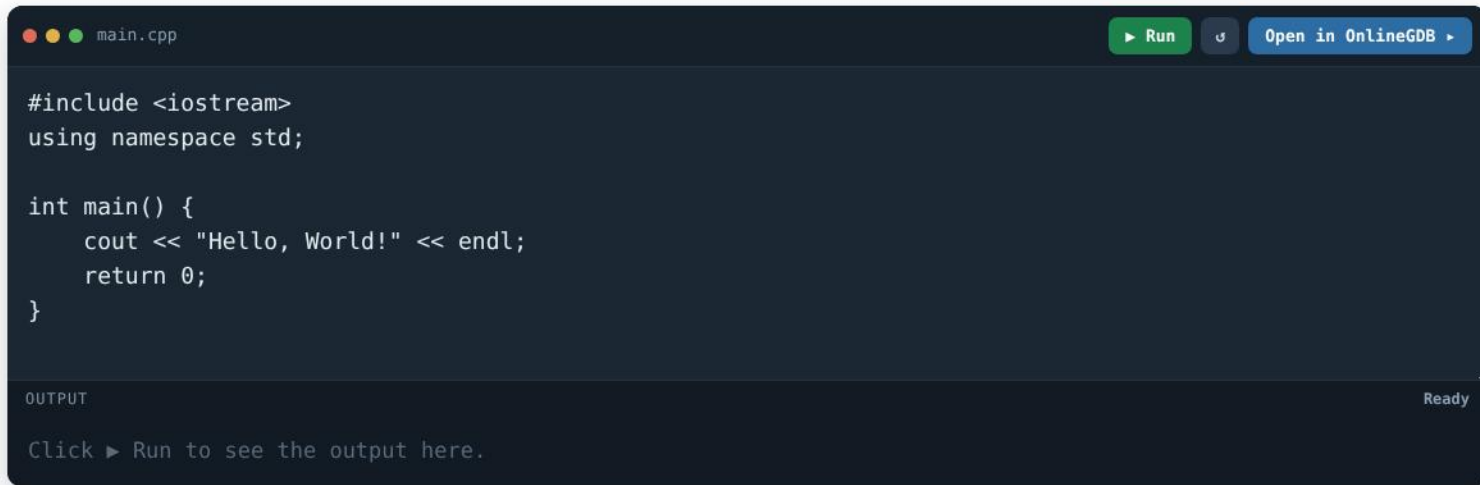


Every statement ends with ;

The semicolon is how C++ knows one instruction has finished. Forgetting it is the #1 beginner slip.

Run your first program

Click ► **Run**. Then change the words inside the quotes and run again.



The screenshot shows a code editor window titled 'main.cpp'. The code is as follows:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

At the top right of the editor, there are three buttons: a green 'Run' button with a play icon, a dark grey button with a refresh icon, and a blue 'Open in OnlineGDB' button with a right-pointing arrow. Below the code editor is an 'OUTPUT' section. It contains the text 'Click ► Run to see the output here.' and the word 'Ready' in the bottom right corner.

DID YOU NOTICE?

A small shortcut: `using namespace std;`

In the live box we wrote `cout`, not `std::cout`. The line `using namespace std;` lets us drop the `std::` prefix.

For beginners

We'll use this shortcut in class to keep code short. Both styles are correct.

What does this print?

```
cout << "C" << "+" << "+";  
cout << " is fun";
```

A C + + is fun (with spaces)

B C++ is fun

C C++ (newline) is fun

PART 3

Mastering output

You decide exactly what appears on the screen.



Chain messages with <<



New lines & tabs: `\n` and `\t`



Experiment live + a fill-in puzzle



OUTPUT

Chaining with <<

```
cout << "Hello" << ", " << "World" << "!" << endl;
```

Send several pieces in one line. They print left to right, joined together.

Prints

Hello, World!

OUTPUT

New lines & special characters

endl

Move to a new line.

\n

Also a new line, inside quotes.

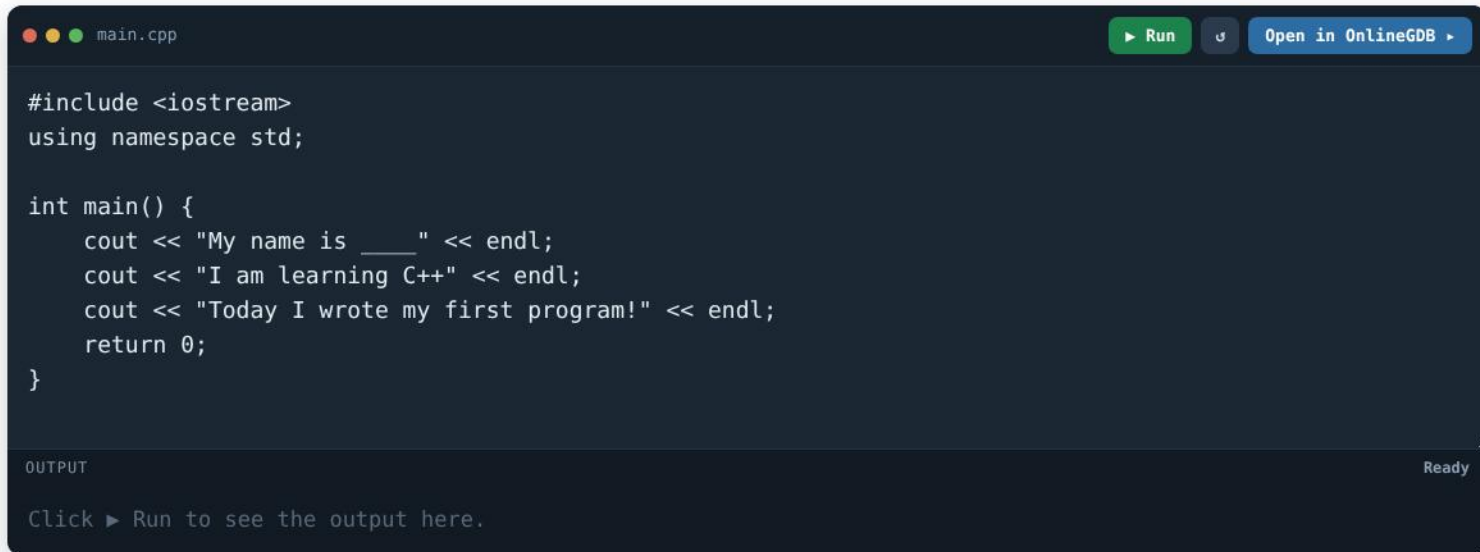
\t

Insert a tab space.

```
cout << "Line one\nLine two\tTabbed" << endl;
```

Make it print your way

Run it. Then try: add a `\n`, add a `\t`, print three lines about yourself.



```
main.cpp

#include <iostream>
using namespace std;

int main() {
    cout << "My name is ____" << endl;
    cout << "I am learning C++" << endl;
    cout << "Today I wrote my first program!" << endl;
    return 0;
}
```

OUTPUT

Click ▶ Run to see the output here.

Ready

Leaving notes: comments

```
1 // A single-line comment – the computer ignores it
2 cout << "Hi"; // comments can sit after code too
3
4 /* A multi-line
5    comment */
```

Comments are for **humans**. The compiler skips them. Use them to explain *why*.

What's missing?

Three blanks turn this into a working program. Call them out.

```
# _____ <iostream>
using namespace std;

int _____ () {
    cout << "Done!" << endl _____
    return 0;
}
```

► Tap to reveal the answer

PART 4

Talking to the user

Programs get far more fun when they listen and respond.



Read input with `cin >>`



Greet the user by name — live



A mini challenge with real logic



INPUT

Reading input with `cin`

```
1 char name[40];  
2 cout << "What is your name? ";  
3 cin >> name;  
4 cout << "Hello, " << name << "!";
```

`cin >>`

Reads what the user types into a variable.

The arrows flip

`cout <<` sends out · `cin >>` brings in.

Greet the user by name

The input box is pre-filled with a sample name. Run it — then change the greeting and run again.



The screenshot shows a C++ IDE window titled 'main.cpp'. The code defines a `main` function that prompts the user for their name and prints a personalized welcome message. The IDE includes a 'Run' button and a link to 'Open in OnlineGDB'. Below the code editor is an 'OUTPUT' section that is currently empty, with a message to click 'Run' to see the output.

```
#include <iostream>
using namespace std;

int main() {
    char name[40];
    cout << "What is your name? ";
    cin >> name;
    cout << "Welcome, " << name << "!" << endl;
    return 0;
}
```

OUTPUT Ready

Click ▶ Run to see the output here.

Add a second question

The program reads a name and a favorite number. Make it also print the number **doubled**. (Hint: `num * 2`)

main.cpp

▶ Run

⌘

Open in OnlineGDB ▶

```
#include <iostream>
using namespace std;

int main() {
    char name[40];
    int num;
    cout << "Name and a number: ";
    cin >> name >> num;
    cout << "Hi " << name << ", your number is " << num << endl;
    // TODO: print num doubled
    return 0;
}
```

OUTPUT

Ready

Click ▶ Run to see the output here.

PART 5

Errors are your friends

Every programmer meets them daily — let's make some on purpose.



The 3 errors you'll meet most



Break a program on purpose — live



How real programmers debug



Three errors you *will* meet



Missing `;`

"expected `';`" — add the semicolon.



Mismatched `{ }`

Every opening brace needs a closing one.

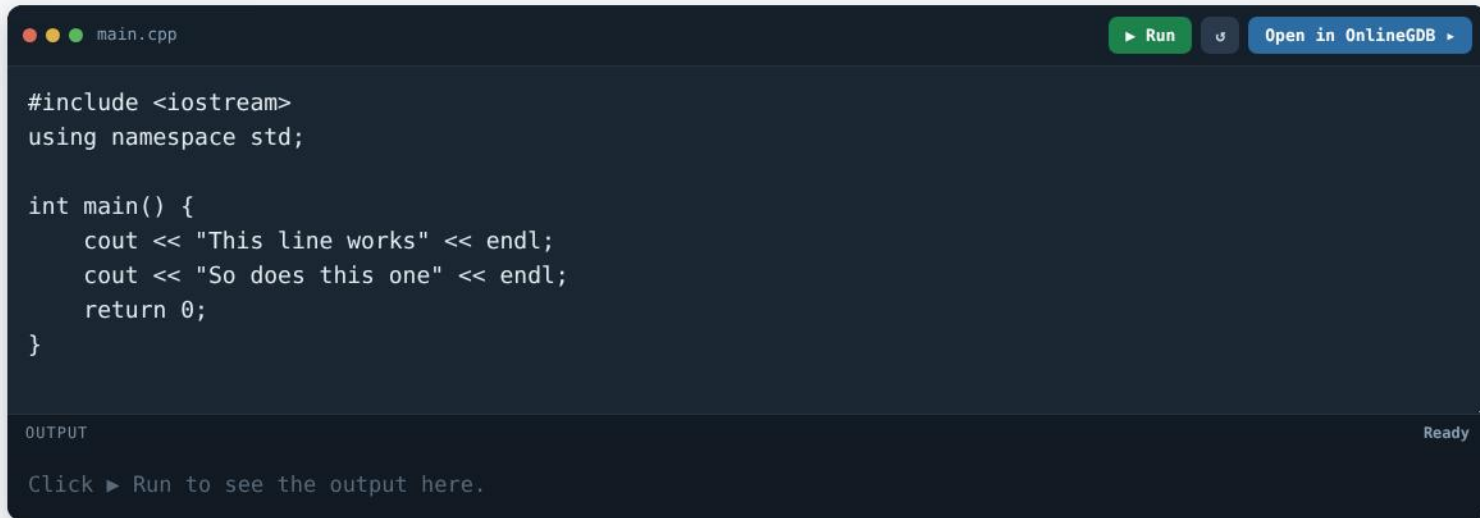


Typo in a name

`cot` instead of `cout`.

Cause an error on purpose

Run it (works). Now **delete the semicolon** after the first line and run again. Read the error. Then put it back.



```
#include <iostream>
using namespace std;

int main() {
    cout << "This line works" << endl;
    cout << "So does this one" << endl;
    return 0;
}
```

OUTPUT

Click ▶ Run to see the output here.

The lesson

Now you've seen that error once — you'll recognize it forever.

MINDSET

How real programmers debug

**Read the message**

It usually names the line and the problem.

**Fix one thing, run again**

Small steps beat big guesses.

**Repeat calmly**

Every bug you fix makes you faster.

PART 6

Let's put it together

Time to think, build, and show what you've learned.



A quick 3-question quiz



The "Smart ID Card" brain challenge



Build with a partner & demo



Where does a C++ program start?

A At the first `#include`

B At the `main` function

C At the first `cout`

What ends almost every statement?

A A full stop .

B A semicolon ;

C A new line

Which sends text to the screen?

A `cout <<`

B `cin >>`

C `print()`

The Smart ID Card

Build a program that **asks** the user, then **figures things out**:

1. Ask for their **name** and their **birth year**.
2. **Think**: compute their **age in 2026** — don't let them type the age!
3. **Trickier**: print their **initials** only (hint: first letter of the name).
4. Show it all inside a neat bordered card using `\t`.

15:00

|| Pause



+1 min

The catch

The age must be *calculated* by the program (2026 – birth year), not entered. That's the difference between a printout and a program.

Your Smart ID Card

Starter reads the data. **You add** the age calculation and the card. (Input box is pre-filled: Sara 2005)

main.cpp

▶ Run

↶

Open in OnlineGDB ▶

```
#include <iostream>
using namespace std;

int main() {
    char name[40];
    int birthYear;
    cout << "Name and birth year: ";
    cin >> name >> birthYear;

    // TODO 1: compute age -> int age = 2026 - birthYear;
    // TODO 2: print a bordered card with name, age, and first initial

    cout << "=====" << endl;
    cout << "  Name: " << name << endl;
    // add age and initial lines here
    cout << "=====" << endl;
    return 0;
}
```

How the smart card works

```
int age = 2026 - birthYear;    // computed!

cout << "=====" << endl;
cout << "  Name:   " << name << endl;
cout << "  Age:    " << age  << endl;
cout << "  Initial: " << name[0] << endl;
cout << "=====" << endl;
```



Age is derived

The program does the maths, not the user.



name[0] = first letter

Text is made of characters you can pick out.



That's real logic

Input → process → output. The heart of every program.

PROJECT CONNECTION

You just built a welcome screen

Almost every program starts with a header that greets the user — whatever project we choose as a class, that intro card is step one.

Next lecture

we add **variables**, so the program can ask the user's real name and remember it.

What should *we* build this semester?

Talk it through as a class. A good first project is:

1. **Useful** — something you'd actually want to use
2. **Simple to start** — runs in the console, text in / text out
3. **Growable** — we can add a feature each week

Ideas: a quiz game · a budget tracker · a to-do list · a simple bank · a mini library · a score keeper...



We'll pick one together and build it step by step.

What you learned today



Concepts

A program is precise instructions; a language + compiler turns them into something that runs.



Skills

You wrote, ran, broke, and fixed real C++ — and printed your own output with `cout`.

Milestone

You are now a programmer. Everything ahead builds on today.

KEEP THESE WORDS

Today's vocabulary

#include

Load a library

main

Where it starts

cout

Print to screen

cin

Read input

;

Ends a statement

{ }

Group of code

endl

New line

//

A comment

BEFORE NEXT TIME

Practice (15–20 min)

**Write three small programs**

A short poem, your three favorite foods, and a picture made from text symbols.

**Break one on purpose**

Remove a `;`, read the error, put it back.

**Save everything**

Start a folder for your code — your growing library.

Lecture 2 — Variables & Data Types

Today you printed fixed text. Next time your programs will **remember things** and work with real data.

- Store numbers, text, and more in variables
- Proper text with `string`
- Make the intro card ask for the user's real name





You wrote your first programs.

That's the hardest step — and you've taken it. See you next lecture.

Dr. Mir Hassan · Associate Professor in AI & Trustworthy Digital Systems